

FEATURE OVERVIEW

Frappier is completed project template with all features which are commonly used in projects.

Technologies ASP.NET Core 3.1, Entity Framework Core 3.1, jQuery, Bootstrap, SQL Server

Feature Overview

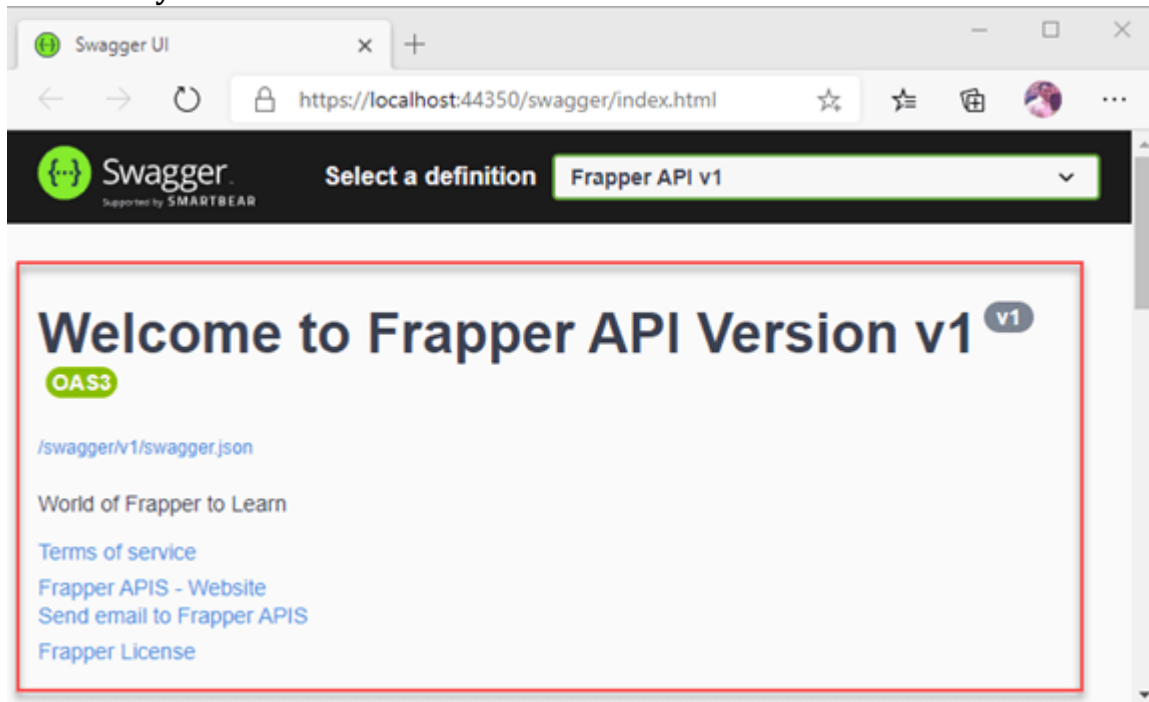
- Swagger
- Versioning
- Using JWT Token
- CRUD Operation
- Paging
- Validation
- Custom Middleware
- Request and Response Logging
- Exception Handling and Error Logging
- Common Response
- Using Unit of Work with Entity Framework Core and Dapper
- How to configure and run
- Clone code from Github: [git clone Link](#)
- Open solution Frappier.API.sln in Visual Studio 2019
- Database Scripts FrappierAPIDB (Main Database), [Download Database Script](#)
- Run Database Script which is provided
- appsettings.json file update DatabaseConnection (FrappierAPIDB Database)
- Make Changes in ConnectionStrings in appsettings.json file
- Build project which will restore all NuGet Packages
- Final Step Run Project

Swagger

For Designing API documentation, we have Used Swagger, If you see below Swagger documentation, you will find many details about these APIs such as Name, Url, Email, Version, Title, Description, terms of service, Contact, License.

What is Swagger?

Simplify API development for users, teams, and enterprises with the Swagger open source and professional toolset. Find out how Swagger can help you design and document your APIs at scale.



Version

To do versioning in ASP.NET Core WEBAPI, We have installed package from NuGet **Microsoft.AspNetCore.Mvc.Versioning**

After installing packages, we are going to register **AddApiVersioning** Service in **ConfigureServices** class.

```
services.AddApiVersioning(x:ApiVersioningOptions =>
{
    x.DefaultApiVersion = new ApiVersion(majorVersion:1, minorVersion:0);
    x.AssumeDefaultVersionWhenUnspecified = true;
    x.ReportApiVersions = true;
    x.ApiVersionReader = new HeaderApiVersionReader(params headerNames: "x-frapper-api-version");
});
```

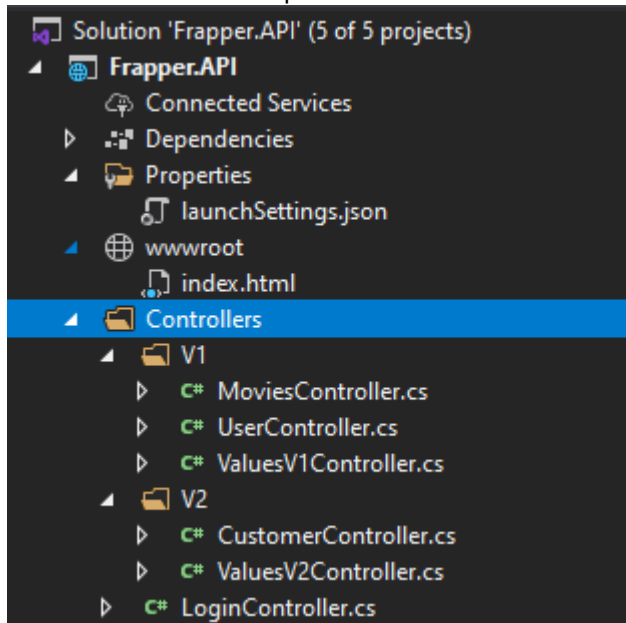
After registering Service, we can use **ApiVersion** Attribute and **MapToApiVersion** Attribute and specify a version.

```
5 namespace Frapper.API.Controllers.V1
6 {
7     [Authorize(Roles = "User")]
8     [Route("api/values")]
9     [ApiVersion("1.0", Deprecated = true)]
10    [ApiController]
11    References
12    public class ValuesV1Controller : ControllerBase
13    {
14        // GET: api/<ValuesController>
15        [HttpGet]
16        [MapToApiVersion("1.0")]
17        References
18        public IEnumerable<string> Get()
19        {
20            return new string[] { "value1", "value2" };
21        }
22    }
```

```
10 namespace Frapper.API.Controllers.V2
11 {
12     [Authorize(Roles = "User")]
13     [Route("api/values")]
14     [ApiVersion("2.0")]
15     [ApiController]
16     References
17     public class ValuesV2Controller : ControllerBase
18     {
19         // GET: api/<ValuesController>
20         [HttpGet]
21         [MapToApiVersion("2.0")]
22         References
23         public IEnumerable<string> Get()
24         {
25             return new string[] { "value3", "value4" };
26         }
27     }
```

Folder Structure

We have created a separate folder for a different version of APIs.



Using JSON Web Token (JWT)

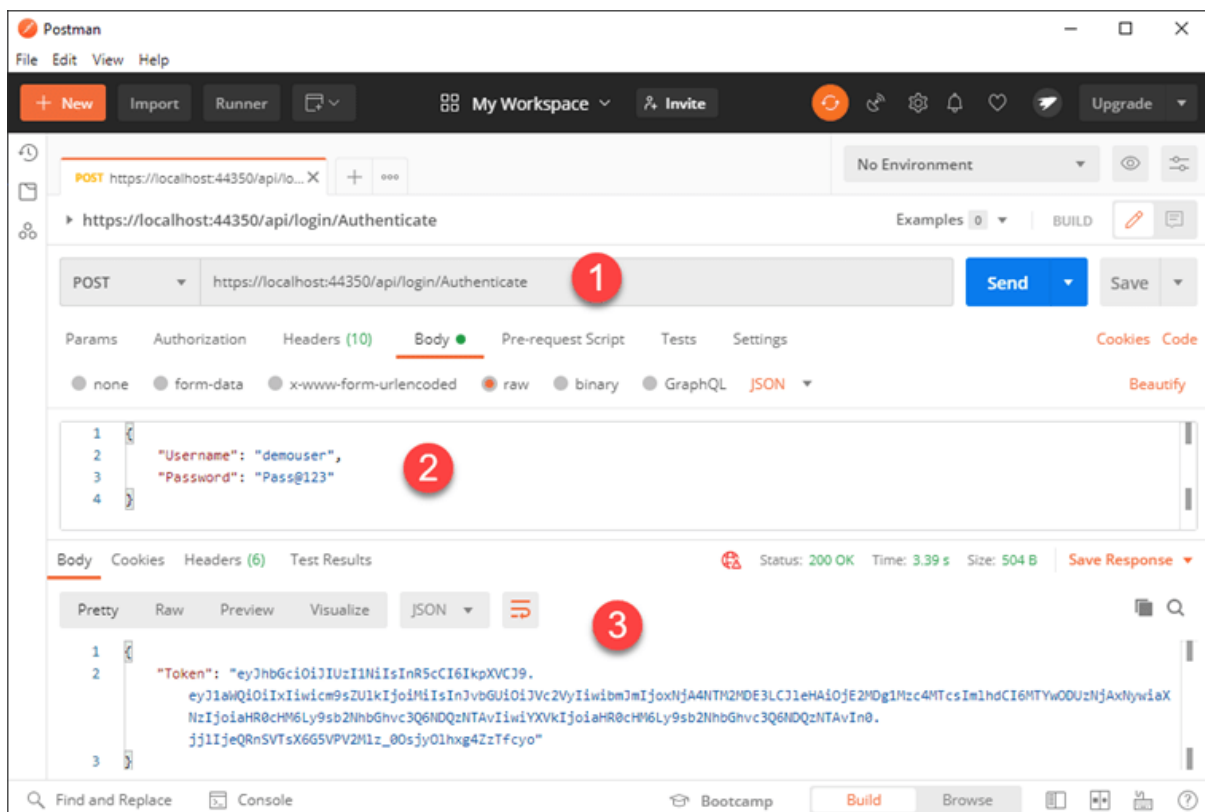
Authentication

In this part, we will see Authentication in details we are going pass Username and Password if credentials are valid then in return, we will receive json web token (JWT token), We are going to create User and share their credentials to access our APIs.

Results

Messages

	UserId	UserName	FirstName	LastName	EmailId	MobileNo	Gender	Status	CreatedBy
1	1	demouser	Saineshwar	Bageri	demouser@frapper.com	9999999999	M	1	1
2	2	demoadmin	demoadmin	demoadmin	demoadmin@frapper.com	9998889998	M	1	1

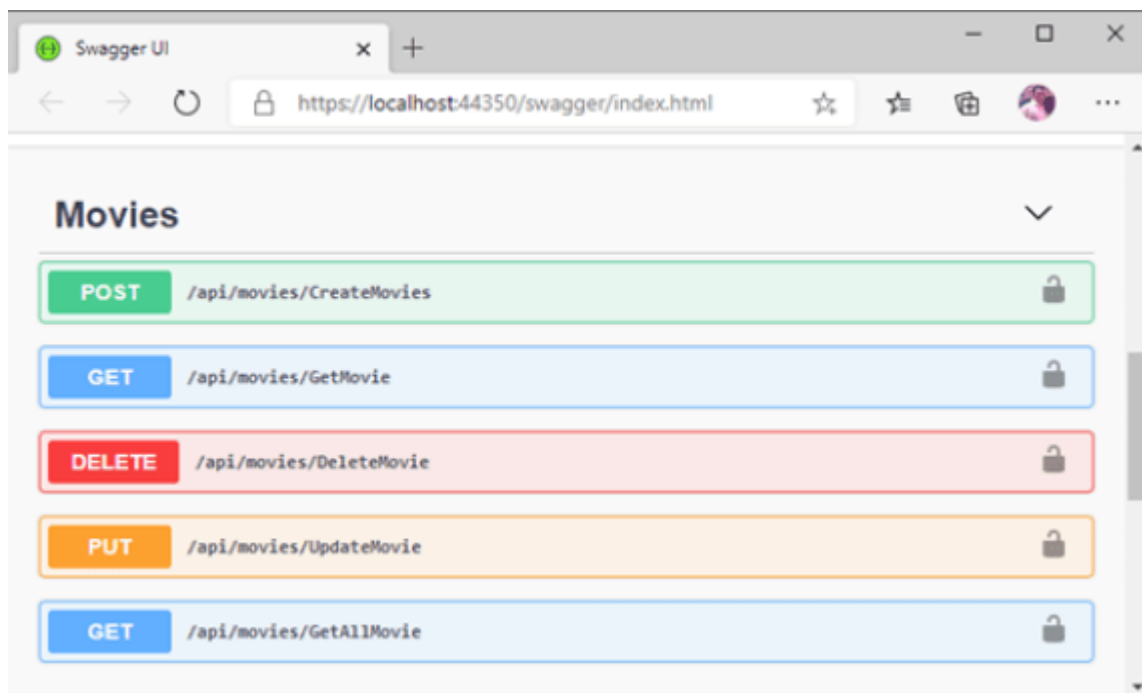


After getting token now let's use it for performing crud operations.

CRUD Operation

In this part, we are going to perform CRUD operation.

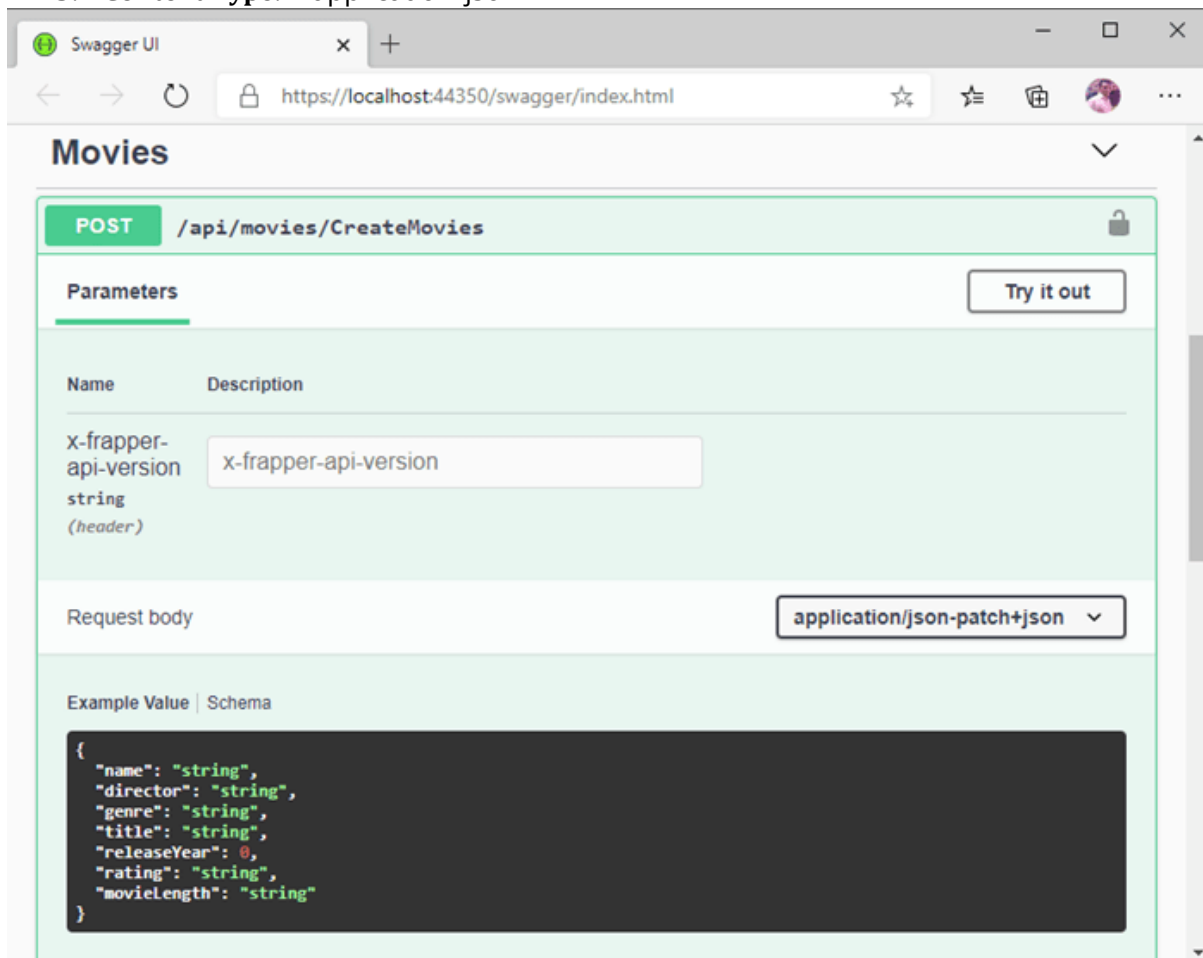
- Creating Movie
- Getting Movie Details by Id
- Deleting Movie Details
- Updating Movie Details
- Get All Movies



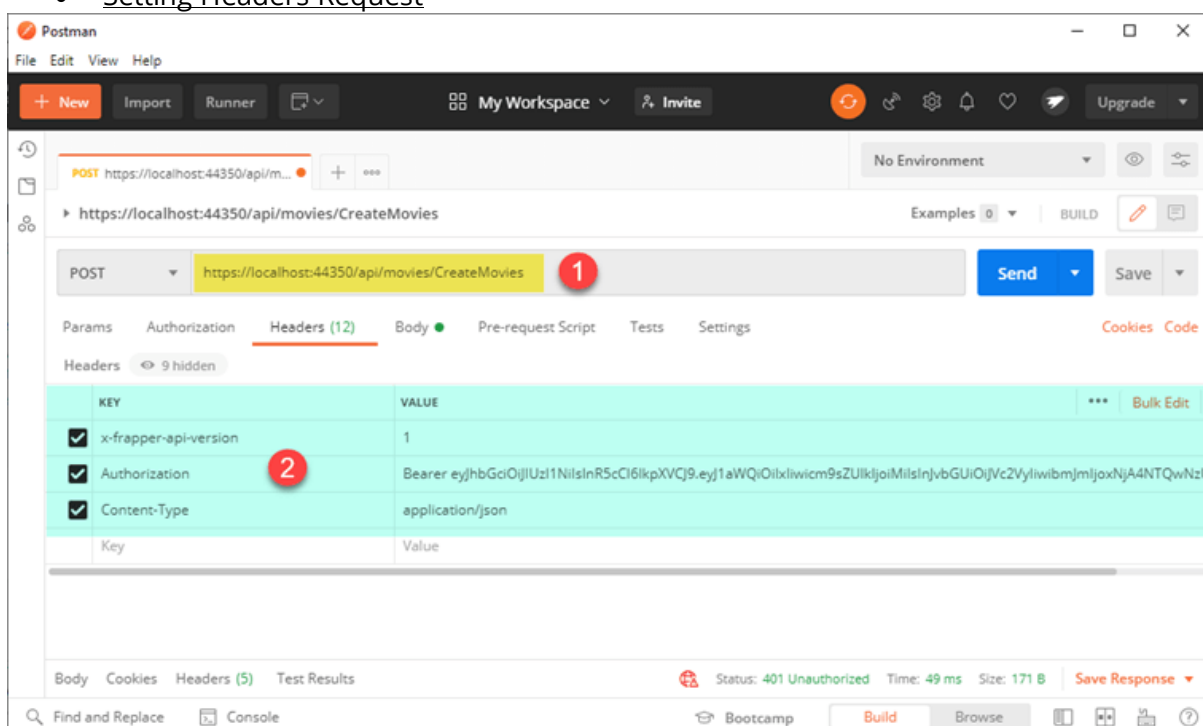
Creating Movie

For creating Movie, we need to send below headers

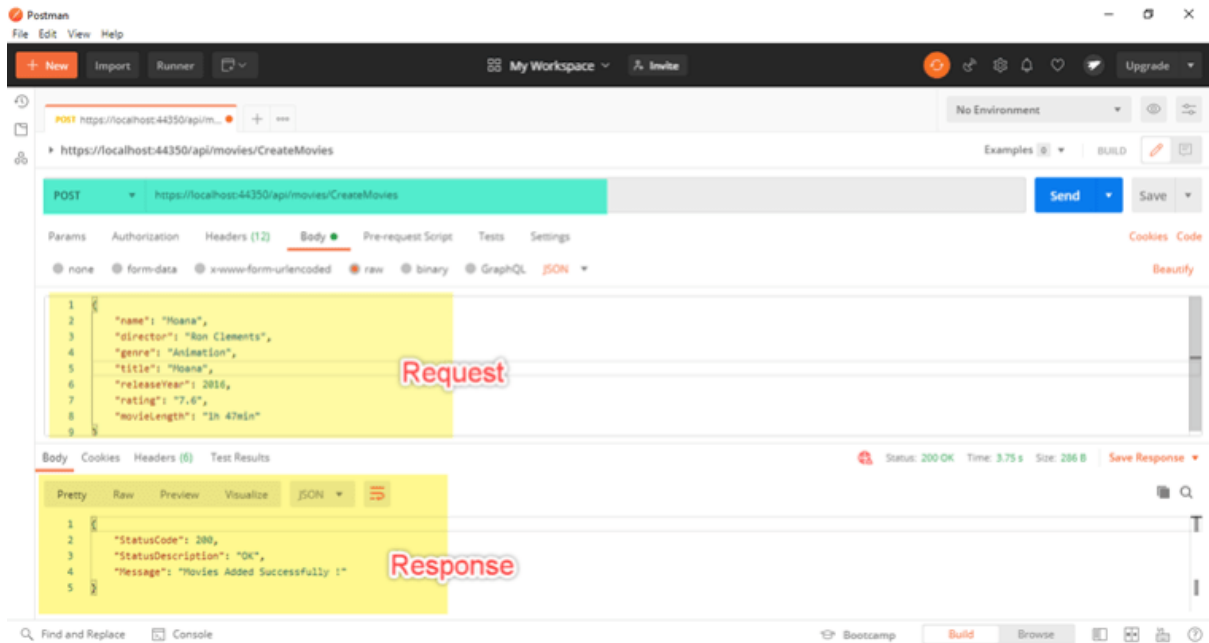
1. **x-frapper-api-version** : – Version of API 1 or 2
2. **Authorization:** – Jwt token
3. **Content Type:** – application/json



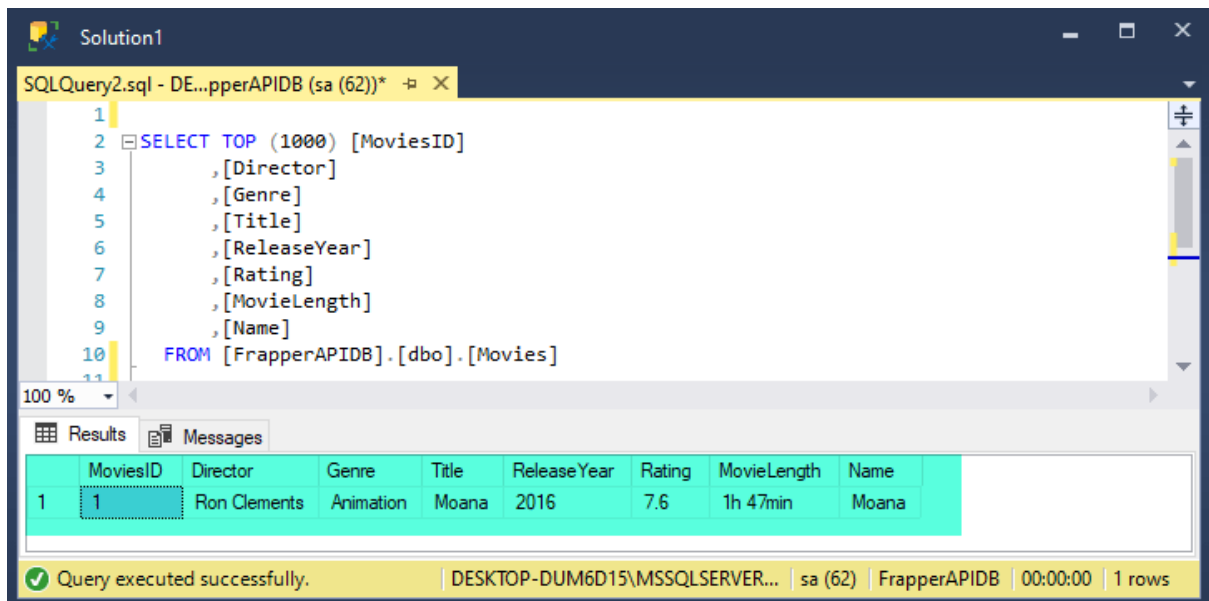
- Setting Headers Request



Response



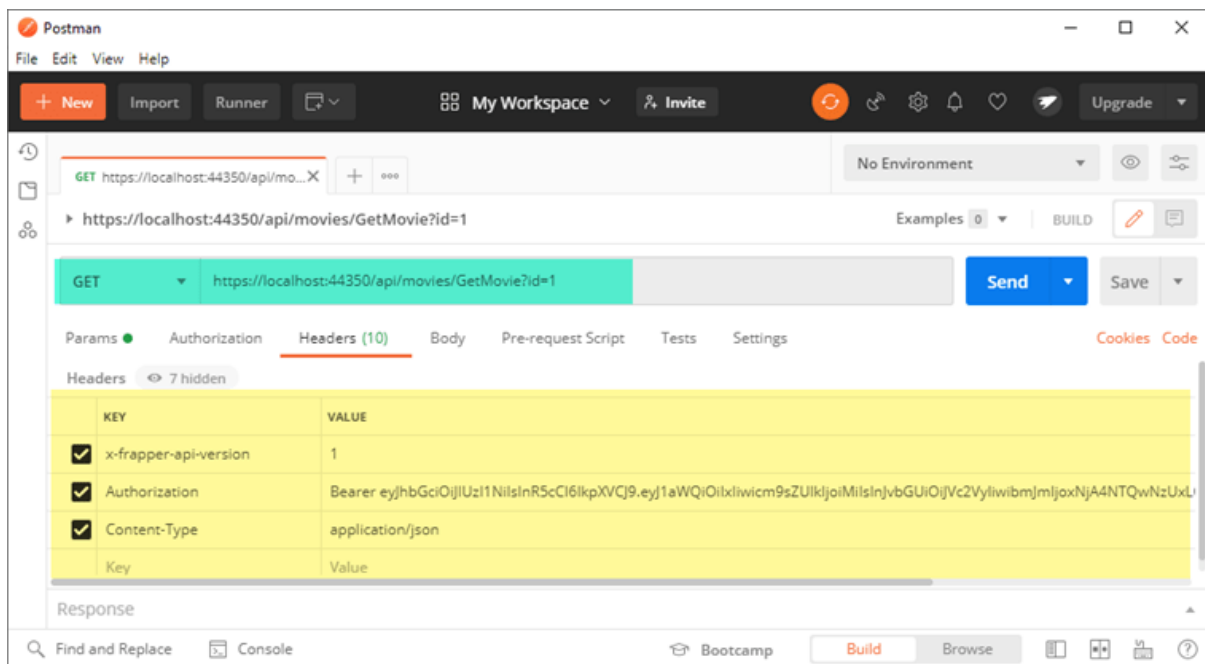
Output



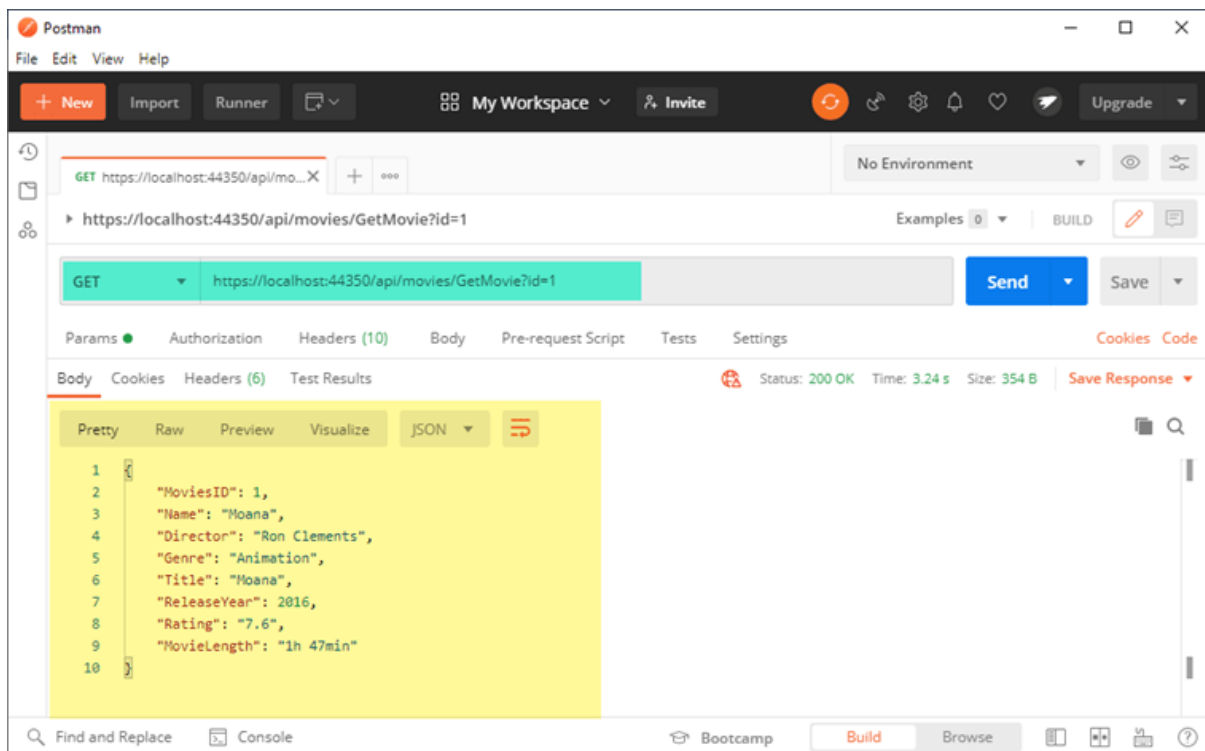
Get Movie Details by Id

In this part, we will get Movie details that we have created by sending Id (**MoviesID**). While getting details also we need to send Headers which we have sent while creating Movie.

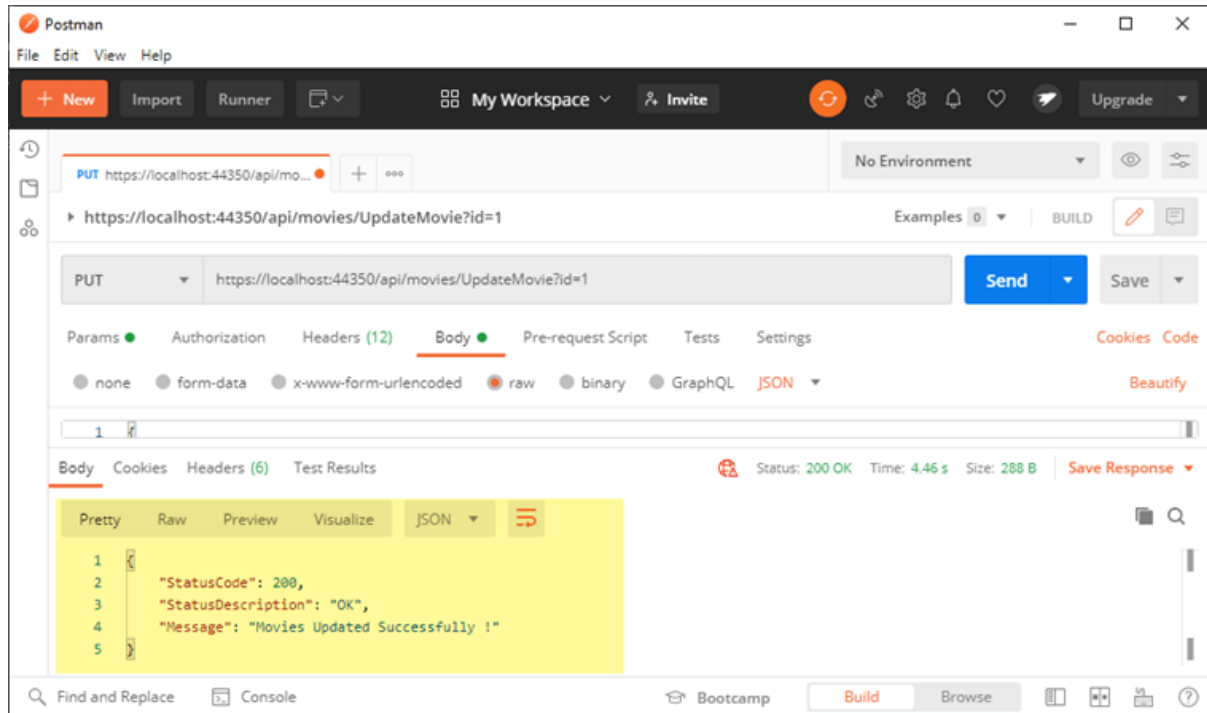
Request



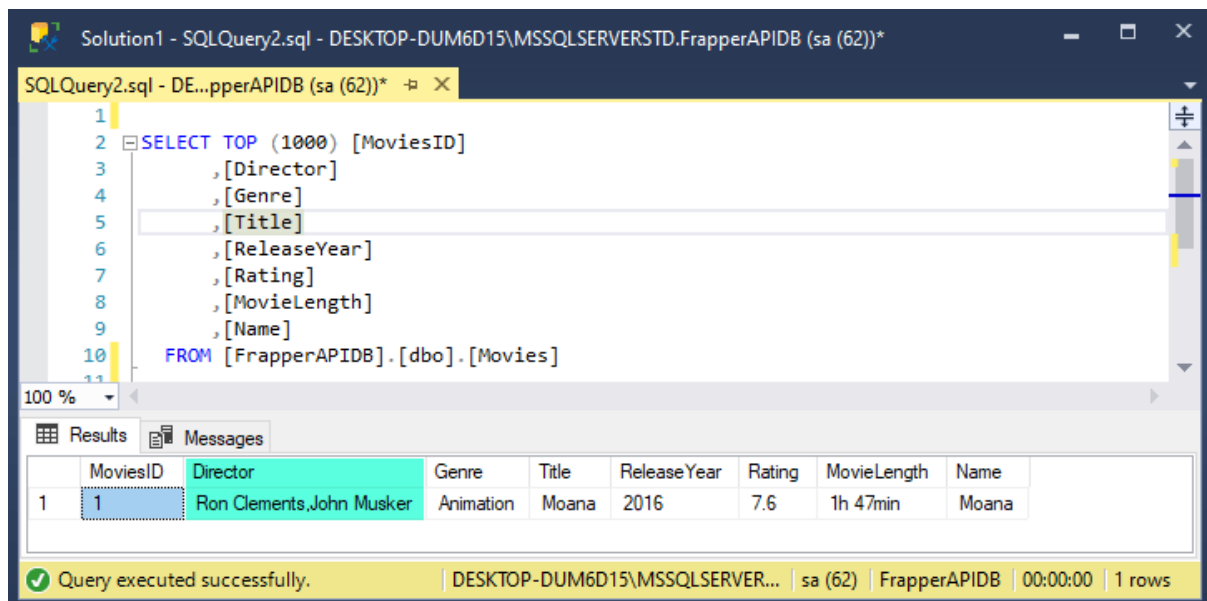
Response



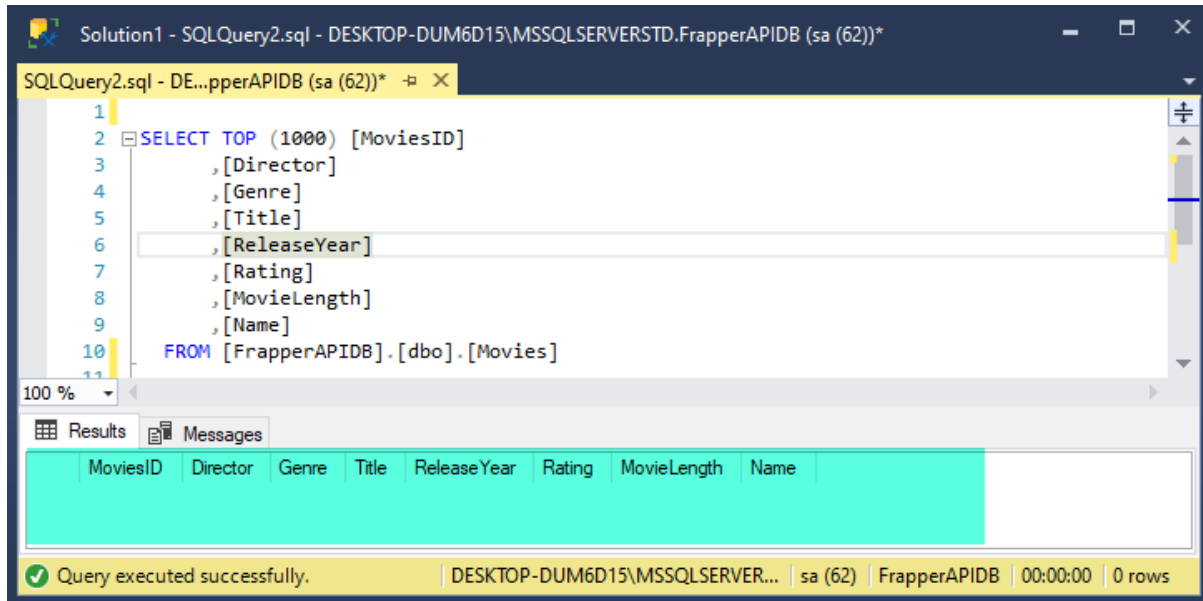
Response



Output



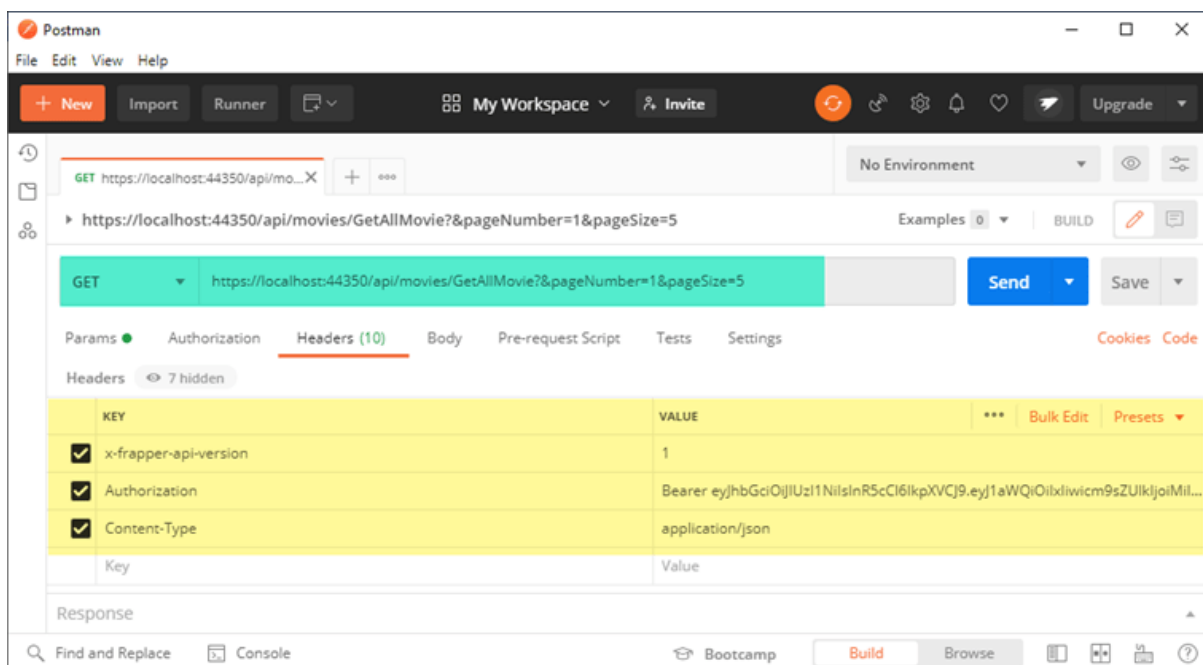
Output



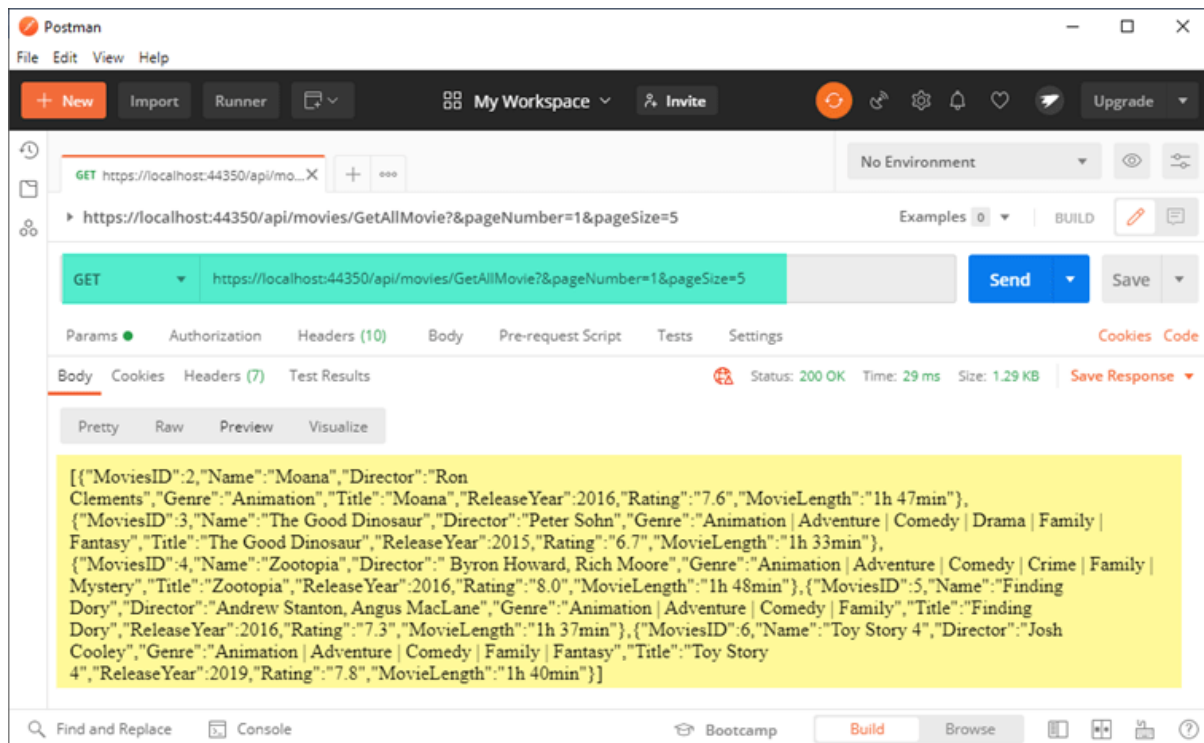
Paging

In this part, we will get all Movies Details for doing that we will use paging where we are going to send page number and pagesize parameters in URL according to it we are going get details.

Request



Response



Validation

In creating the Movie Model, we have kept all fields mandatory. While we are sending request; we need all fields if it is not there then it will show an error message. For validating we have created ValidateModel Attribute which is an ActionFilter Attribute in this Attribute we are going validating model state if it is invalid then we are returning errors.

```
using System.ComponentModel.DataAnnotations;
namespace Frapper.ViewModel.Movies.Request
{
    public class MoviesCreateRequest
    {
        [Required(ErrorMessage = "Enter Name")]
        [Display(Name = "Name")]
        public string Name { get; set; }

        [Required(ErrorMessage = "Enter Director")]
        [Display(Name = "Director")]
        public string Director { get; set; }

        [Required(ErrorMessage = "Enter Genre")]
        [Display(Name = "Genre")]
        public string Genre { get; set; }

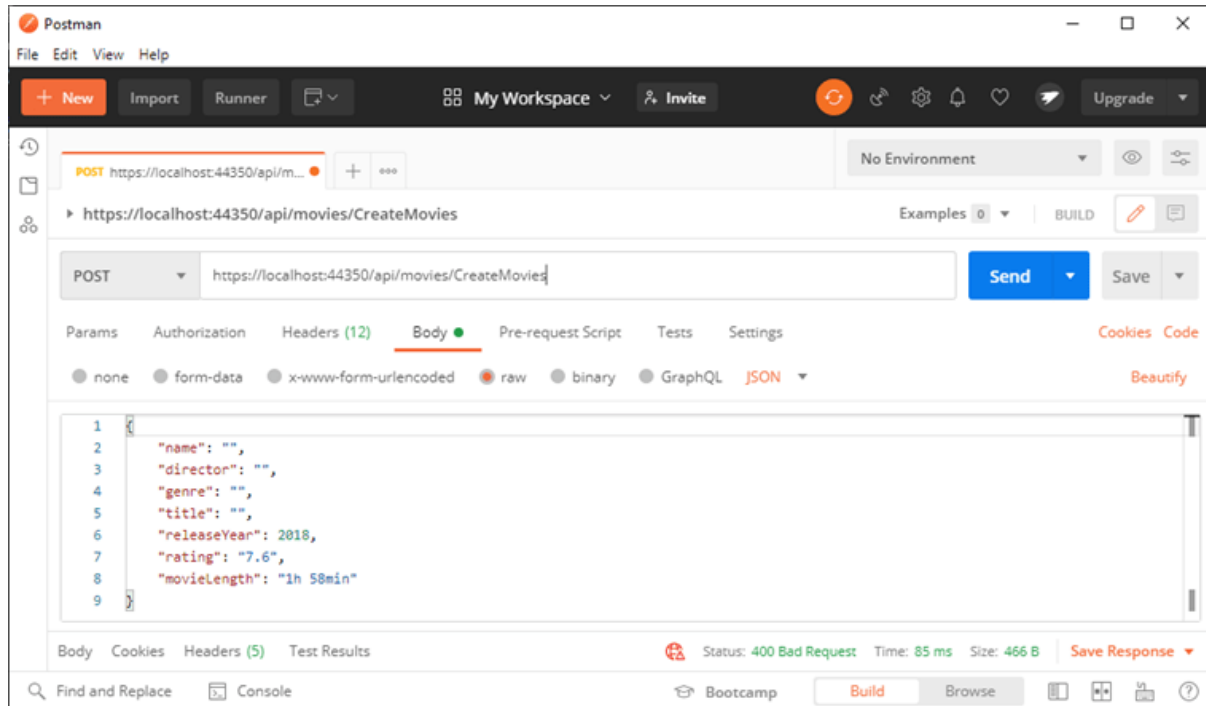
        [Required(ErrorMessage = "Enter Title")]
        [Display(Name = "Title")]
        public string Title { get; set; }

        [Required(ErrorMessage = "Enter ReleaseYear")]
        [Display(Name = "ReleaseYear")]
        public int ReleaseYear { get; set; }

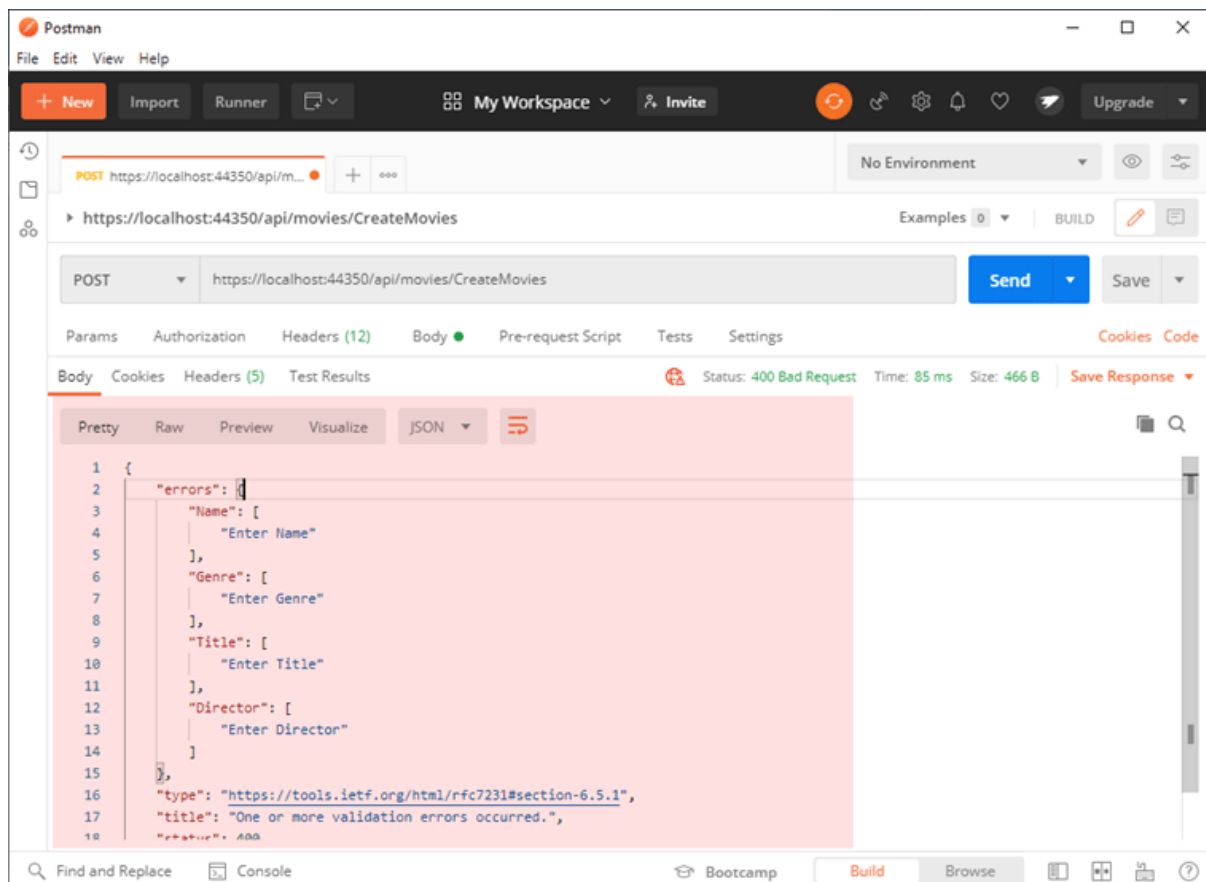
        [Required(ErrorMessage = "Enter Rating")]
        [Display(Name = "Rating")]
        public string Rating { get; set; }

        [Required(ErrorMessage = "Enter MovieLength")]
        [Display(Name = "MovieLength")]
        public string MovieLength { get; set; }
    }
}
```

Request



Response



Custom Middleware

In this application, we have written 2 custom middleware

- Validating Headers
- Logging Request and Response
- Validating Headers

Each request other than Authenticate should have Authorization and API Version header if it is not there, it will show an error.

The screenshot displays two windows. The top window is Visual Studio, showing the code for `ValidateHeaderMiddleware.cs` in the `Frappier.API.Helpers` namespace. The code implements an `Invoke` method that checks for the presence of an `Authorization` header and an `x-frapper-api-version` header. If either is missing, it returns a 401 status code with a corresponding error message. Otherwise, it proceeds to the next middleware in the chain.

```

19
20 public async Task Invoke(HttpContext httpContext)
21 {
22     if (!httpContext.Request.Path.Value.Contains("Authenticate"))
23     {
24         string authorization = httpContext.Request.Headers["Authorization"];
25         if (authorization == null)
26         {
27             httpContext.Response.StatusCode = 401;
28             await httpContext.Response.WriteAsync(text: "Access denied Token Missing!");
29             return;
30         }
31
32         string versionheader = httpContext.Request.Headers["x-frapper-api-version"];
33         if (versionheader == null)
34         {
35             httpContext.Response.StatusCode = 401;
36             await httpContext.Response.WriteAsync(text: "Access denied Version Header Missing!");
37             return;
38         }
39     }
40
41     await _next(httpContext);
42 }
43
44
45

```

The bottom window is Postman, showing a POST request to `https://localhost:44350/api/movies/CreateMovies`. The request headers are configured as follows:

KEY	VALUE
☐ x-frapper-api-version	1
☐ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1aWQiOiJldiwiZm9sZUlkjoiMi...
☒ Content-Type	application/json

The response status is `401 Unauthorized` with the message `Access denied Token Missing!`.

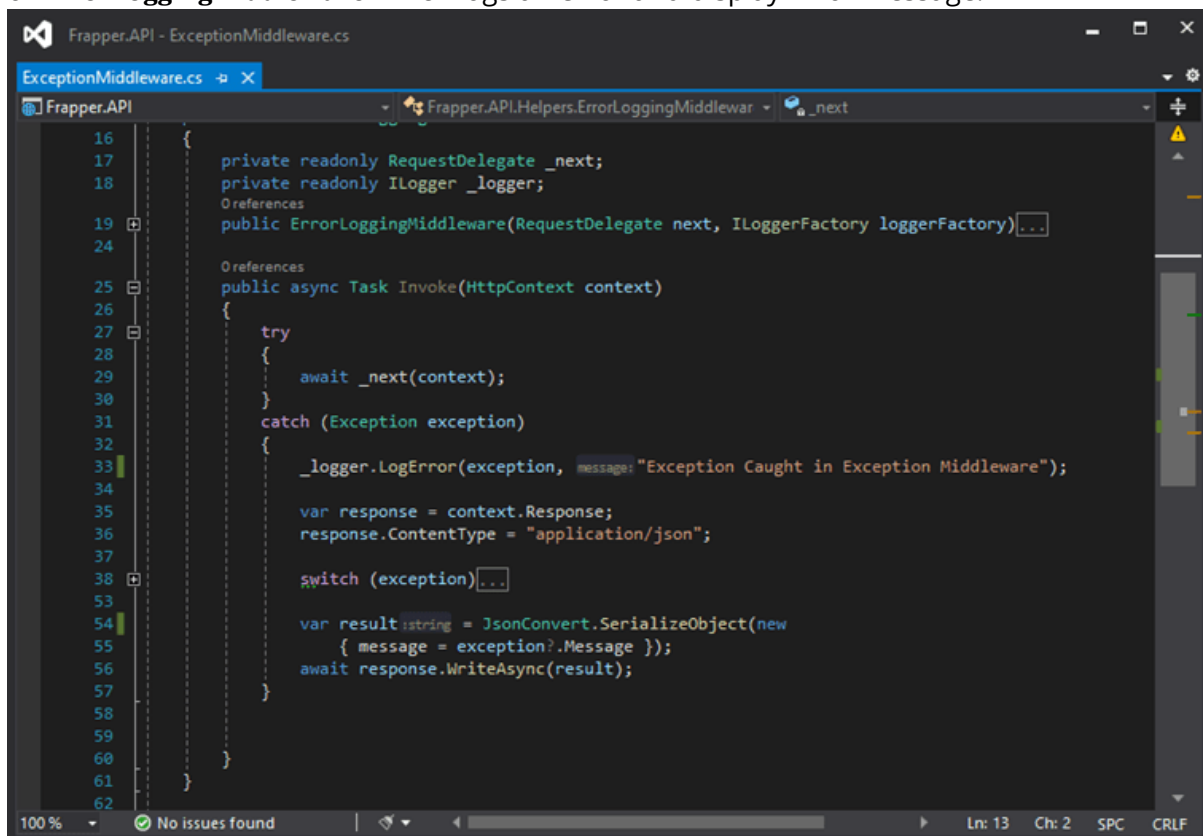
- Logging Request and Response

Logging Request and Response is also middleware which keeps logging each request.

[illegible]

Exception Handling and Error Logging

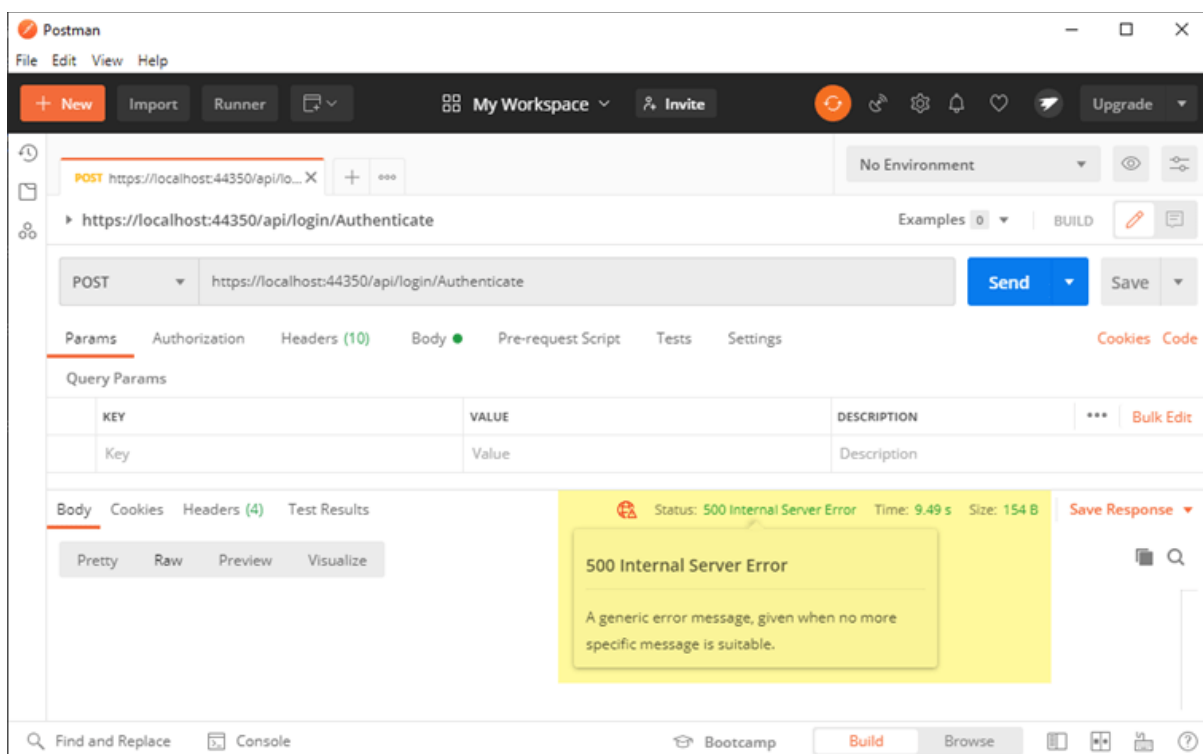
For exception handling and logging error, we have used middleware. Below is an example of **ErrorLoggingMiddleware** which logs an error and display Error Message.



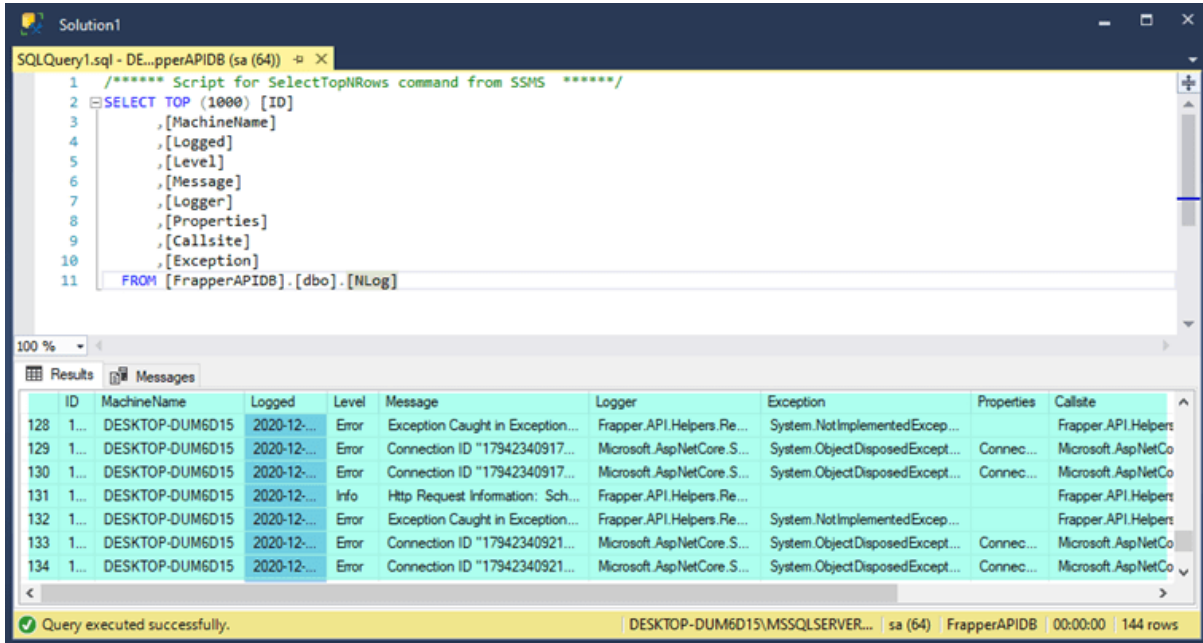
```

16
17     private readonly RequestDelegate _next;
18     private readonly ILogger _logger;
19     References
20     public ErrorLoggingMiddleware(RequestDelegate next, ILoggerFactory loggerFactory)
21     {
22     }
23     References
24     public async Task Invoke(HttpContext context)
25     {
26     try
27     {
28         await _next(context);
29     }
30     catch (Exception exception)
31     {
32         _logger.LogError(exception, "Exception Caught in Exception Middleware");
33
34         var response = context.Response;
35         response.ContentType = "application/json";
36
37         switch (exception)
38         {
39         }
40
41         var result = JsonConvert.SerializeObject(new
42         {
43             message = exception?.Message
44         });
45         await response.WriteAsync(result);
46     }
47 }
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
  
```

If you see below the POST window, we have sent a request that throws an error, but we do not show error details to the user but show standard status code.



Displaying Logged Errors in Database

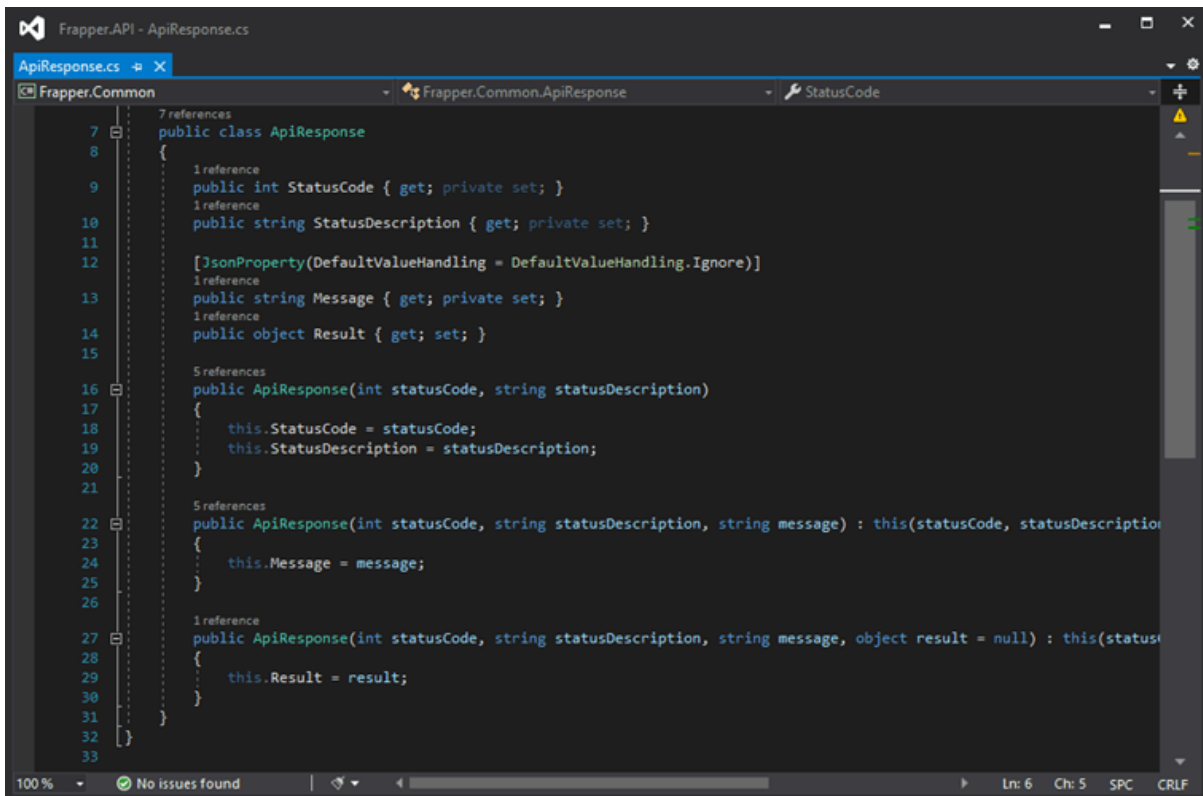


The screenshot shows a SQL query executed in SQL Server Enterprise Manager. The query is a SELECT TOP (1000) statement from the [FrapperAPI DB].[dbo].[NLog] table, listing columns: ID, MachineName, Logged, Level, Message, Logger, Exception, Properties, and CallSite. The results are displayed in a table with 144 rows. The status bar at the bottom indicates the query was executed successfully on the server 'DESKTOP-DUM6D15\MSSQLSERVER...' using the 'sa' user, with a duration of 00:00:00.

ID	MachineName	Logged	Level	Message	Logger	Exception	Properties	CallSite
128	DESKTOP-DUM6D15	2020-12-...	Error	Exception Caught in Exception...	Frappier.API.Helpers.Re...	System.NotImplementedExcep...		Frappier.API.Helpers
129	DESKTOP-DUM6D15	2020-12-...	Error	Connection ID "17942340917..."	Microsoft.AspNetCore.S...	System.ObjectDisposedExcept...	Connec...	Microsoft.AspNetCo
130	DESKTOP-DUM6D15	2020-12-...	Error	Connection ID "17942340917..."	Microsoft.AspNetCore.S...	System.ObjectDisposedExcept...	Connec...	Microsoft.AspNetCo
131	DESKTOP-DUM6D15	2020-12-...	Info	Http Request Information: Sch...	Frappier.API.Helpers.Re...			Frappier.API.Helpers
132	DESKTOP-DUM6D15	2020-12-...	Error	Exception Caught in Exception...	Frappier.API.Helpers.Re...	System.NotImplementedExcep...		Frappier.API.Helpers
133	DESKTOP-DUM6D15	2020-12-...	Error	Connection ID "17942340921..."	Microsoft.AspNetCore.S...	System.ObjectDisposedExcept...	Connec...	Microsoft.AspNetCo
134	DESKTOP-DUM6D15	2020-12-...	Error	Connection ID "17942340921..."	Microsoft.AspNetCore.S...	System.ObjectDisposedExcept...	Connec...	Microsoft.AspNetCo

Common Response

We have tried to create a common response in an entire application for that we have created base Class ApiResponse. ApiResponse class is inherited by different class such as **OkResponse**, **BadRequestResponse**, **NotFoundResponse**. While sending a response, we use this object to create a common response



The screenshot shows the code for the ApiResponse class in the Frapper.Common namespace. The class is a base class for other response classes. It has properties for StatusCode, StatusDescription, Message, and Result. It also has three constructors: one for StatusCode and StatusDescription, one for StatusCode, StatusDescription, and Message, and one for StatusCode, StatusDescription, Message, and Result.

```

7 references
public class ApiResponse
8
9     1 reference
    public int StatusCode { get; private set; }
    1 reference
    public string StatusDescription { get; private set; }
10
11     [JsonProperty(DefaultValueHandling = DefaultValueHandling.Ignore)]
12     1 reference
    public string Message { get; private set; }
    1 reference
    public object Result { get; set; }
13
14
15
16 5 references
    public ApiResponse(int statusCode, string statusDescription)
17  {
18     this.StatusCode = statusCode;
19     this.StatusDescription = statusDescription;
20 }
21
22 5 references
    public ApiResponse(int statusCode, string statusDescription, string message) : this(statusCode, statusDescription)
23  {
24     this.Message = message;
25 }
26
27 1 reference
    public ApiResponse(int statusCode, string statusDescription, string message, object result = null) : this(statusCode, statusDescription, message)
28  {
29     this.Result = result;
30 }
31
32
33
  
```

```
if (_userMasterQueries.CheckUserExists(createUserRequest.UserName))
{
    return BadRequest(error: new BadRequestResponse(message: "Entered Username Already Exists"));
}

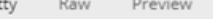
if (_userMasterQueries.CheckEmailExists(createUserRequest.EmailId))
{
    return BadRequest(error: new BadRequestResponse(message: "Entered EmailId Already Exists"));
}

if (_userMasterQueries.CheckMobileNoExists(createUserRequest.MobileNo))
{
    return BadRequest(error: new BadRequestResponse(message: "Entered MobileNo Already Exists"));
}

if (!string.Equals(a: createUserRequest.Password, b: createUserRequest.ConfirmPassword,
StringComparison.Ordinal))
{
    return BadRequest(error: new BadRequestResponse(message: "Password Does not Match"));
}
```

Response

```
1 {
2   "StatusCode": 200,
3   "StatusDescription": "OK",
4   "Message": "Success",
5   "Result": {
6     "Token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1aWQiOiJxIiwicm9sZUlkIjoiaWIsInZvbGUiOiJvc2VyIiwibmJmIjoiaWVjA4IiwjZG9yYyI6eHAiOiJlE2MDg2NDQ0MTIsIm1hdCI6MTYwODY0MzAxMiwiaXNzIjoiaHR0cHM6Ly9sb2NhbGhvc3Q6NDQzNTAvIiwiaXVkiOiJoIiwiaHR0cHM6Ly9sb2NhbGhvc3Q6NDQzNTAvIn0.VkFnSvOk0gXWj1j3b9DhSg_L_Hav3D3R8uP03PYqvPtas"
7   }
8 }
```

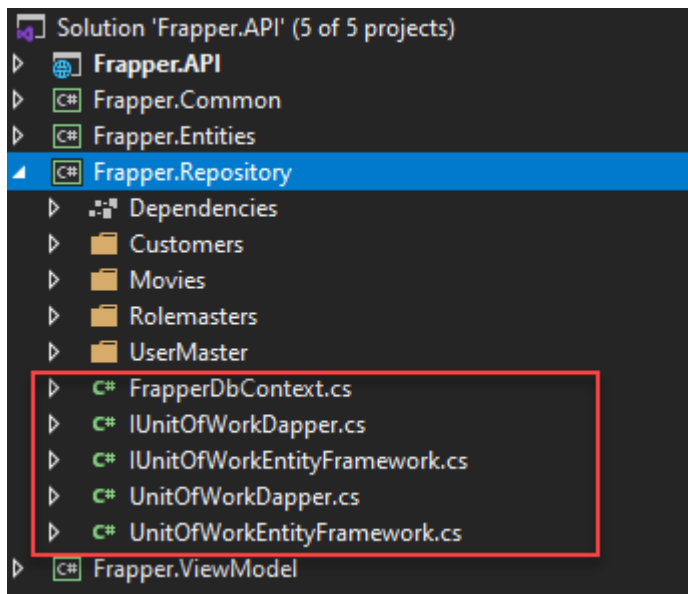


The screenshot shows a web browser's developer console with the 'Pretty' tab selected. A red error message is displayed, indicating a 400 Bad Request. The message states: "Username or password is incorrect".

```
1 2
2  "StatusCode": 404,
3  "StatusDescription": "NotFound",
4  "Message": "Movie Not Found"
5
```

Using Unit of Work with Entity Framework Core and Dapper ORM

In this application, we have used the Unit of Work for Insert, Updating and Deleting record using Entity framework and similar another version of Unit of Work. We have used Dapper ORM for inserting data.



```
[Authorize(Roles = "User")]
[Route(template: "api/movies")]
[ApiVersion("1.0")]
[ApiController]
1 reference
public class MoviesController : ControllerBase
{
    private readonly IUnitOfWorkEntityFramework _unitOfWorkEntityFramework;
    private readonly IMoviesQueries _iMoviesQueries;

    0 references
    public MoviesController(IUnitOfWorkEntityFramework unitOfWorkEntityFramework, IMoviesQueries moviesQueries)
    {
        _unitOfWorkEntityFramework = unitOfWorkEntityFramework;
        _iMoviesQueries = moviesQueries;
    }
}
```

Reference : <https://github.com/saineshwar/Frappier.API>